```
print(df.columns)
```

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   total_bill  244 non-null    float64
 1   tip         244 non-null    float64
 2   sex         244 non-null    category
 3   smoker      244 non-null    category
 4   day         244 non-null    category
 5   time        244 non-null    category
 6   size        244 non-null    int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
None
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')
```

```
#1.Descriptive Statistics
print(df['total_bill'].describe())
```

```
count    244.000000
mean      19.785943
std        8.902412
min        3.070000
25%       13.347500
50%       17.795000
75%       24.127500
max       50.810000
Name: total_bill, dtype: float64
```

```
EDA(Exploratory Data Analysis)is the process of examiming,summarizing and visualizing data before building machine learning models.
```

Why EDA is Important?

1.Understand the structure of the dataset 2.Identify missing values 3.Detect outliers 4.Understand feature distributions 5.Find relationships between variables 6.Check data quality issues 7.Prepare data for ml

Types of EDA

Univariate Analysis

Analysing one variable at a time

Numerical Features

Mean,median,mode,SD,variance,histogram,Box plot

Categorical Features

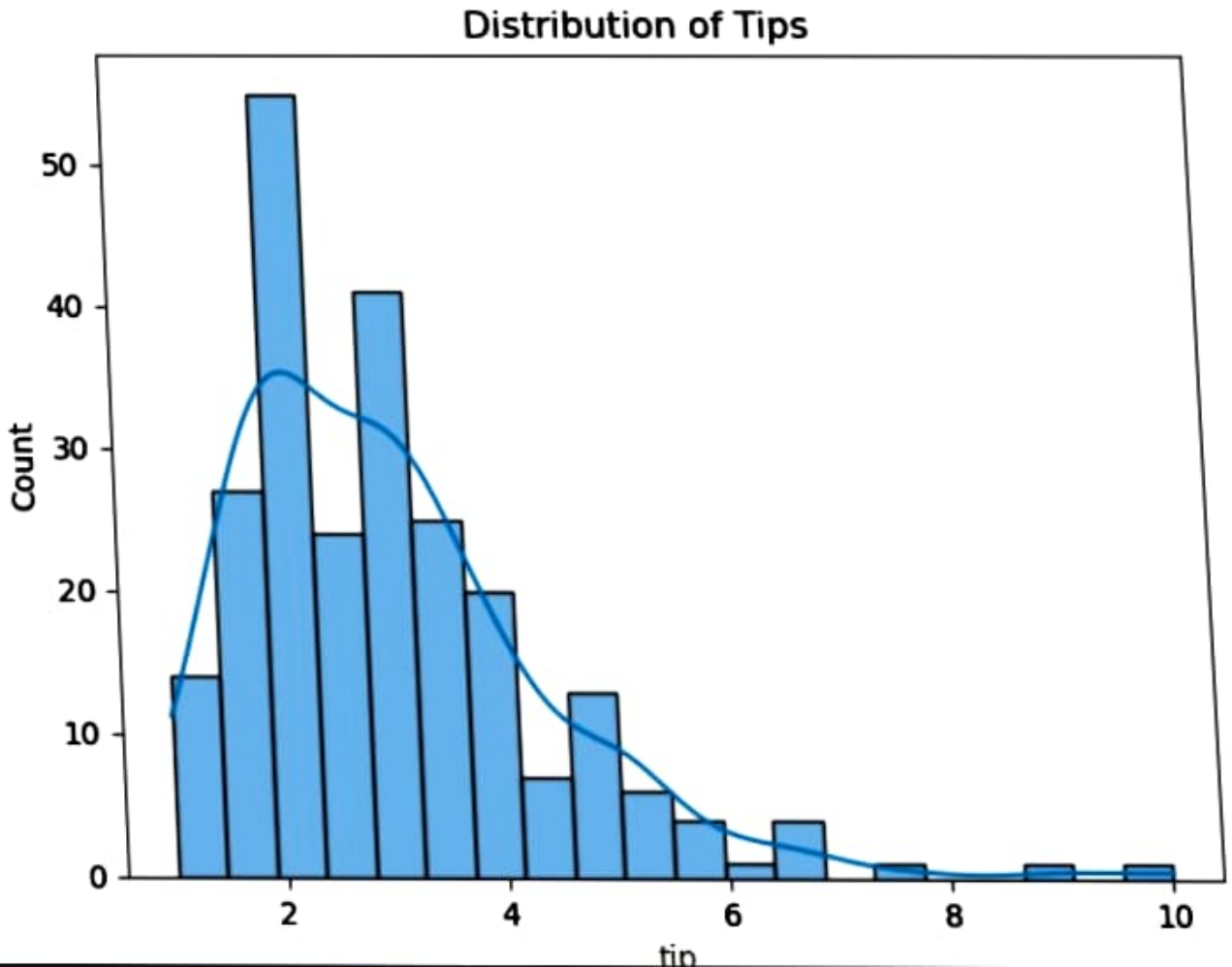
Value counts,Frequency distribution,Bar chart

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df=sns.load_dataset("tips")

print(df.head())
print(df.info())
```

2.Histogram

- A histogram shows the distribution of a numerical feature.
- It didvides data into intervals(bins) and shows how many observations fall into each interval.

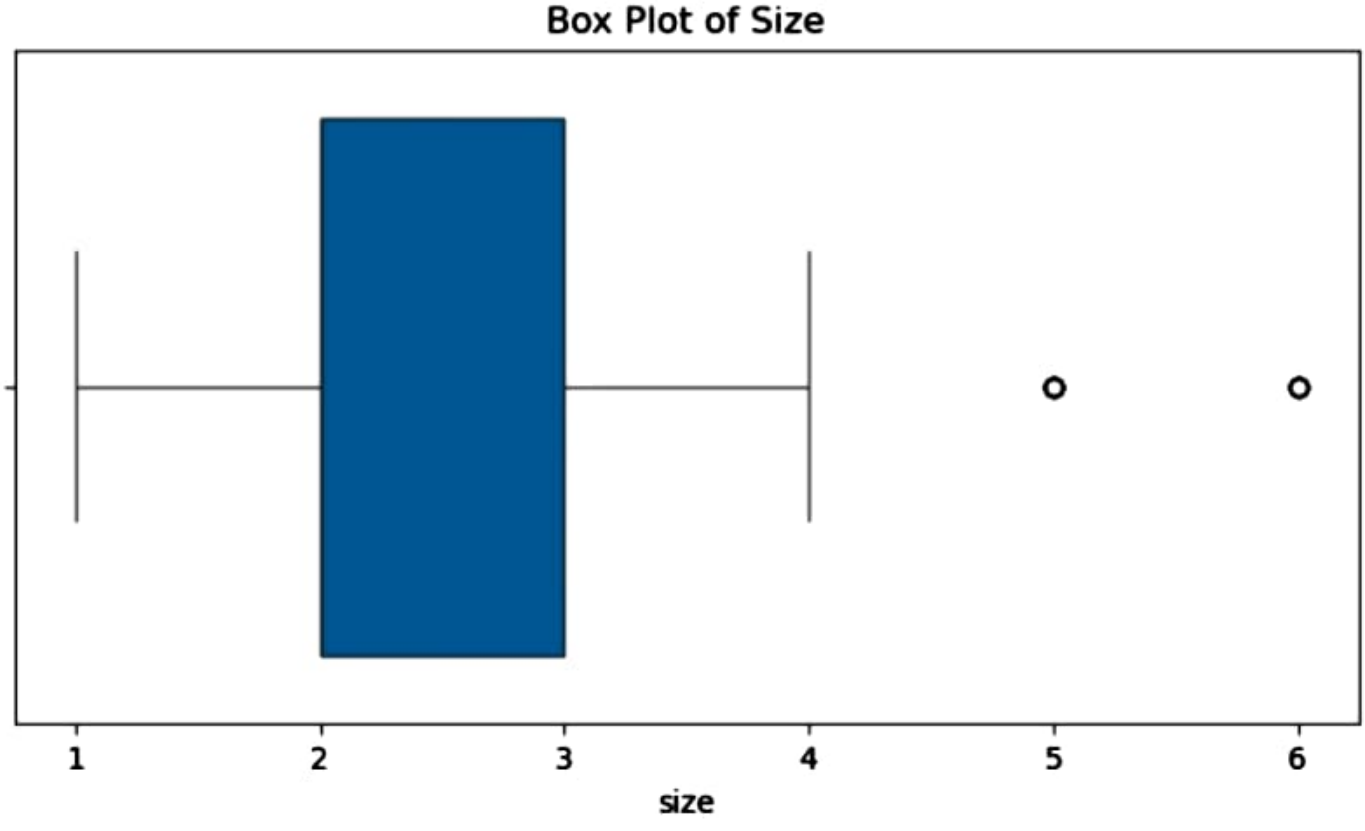
```
sns.histplot(df['tip'],bins=20,kde=True)

plt.title("Distribution of Tips")
plt.show()
```



80
60
40
20
count

3.Box Plot

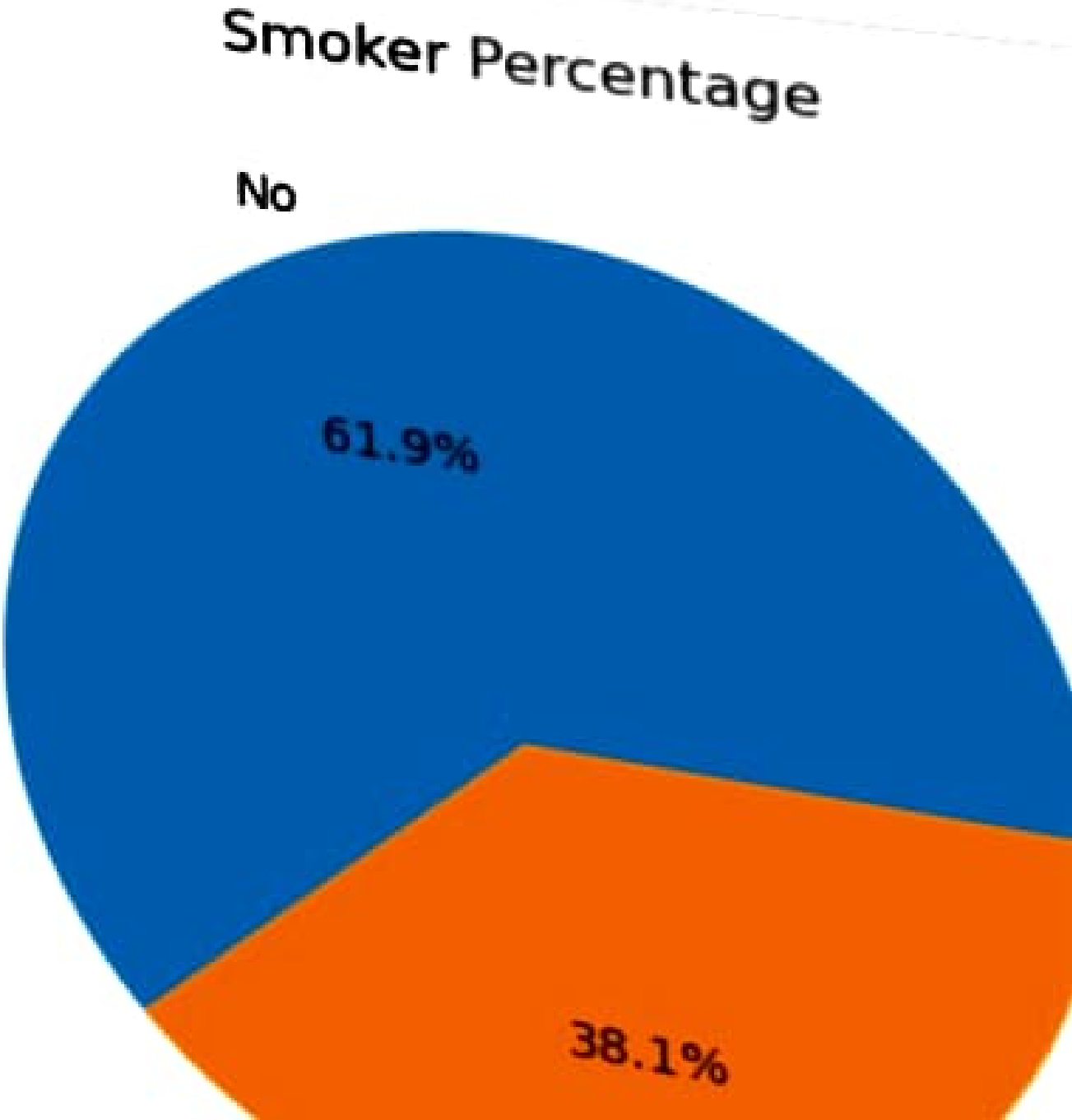
```
plt.figure(figsize=(8,4))
sns.boxplot(x=df['size'])

plt.title("Box Plot of Size")
plt.show()
```



2.Categorical Variable:Day

Value counts

```
#Pie chart
df['smoker'].value_counts().plot(kind='pie',
                                 autopct='%1.1f

plt.title("Smoker Percentage")
plt.ylabel("")                    #to find how
plt.show()
```



```
##Outlier detection using IQR
Q1=df['total_bill'].quantile(0.25)
Q3=df['total_bill'].quantile(0.75)

IQR=Q3-Q1

lower=Q1-1.5*IQR
upper=Q3+1.5*IQR

outliers_iqr=df[(df['total_bill']<lower) | (df['total_bill']>upper)]

print("IQR Outliers:",len(outliers_iqr))
```

```
IQR Outliers: 9
```

Bivariate Analysis

- Bivariant Analysis means analyzing two variables together to understand the relationships between them.
- Bi = Two
- Variate = Variables

Examples

- Study hrs vs Exam Score
- Age vs Salary
- Total bill vs Fare

```
##Outlier detection using IQR
Q1=df['total_bill'].quantile(0.25)
Q3=df['total_bill'].quantile(0.75)

IQR=Q3-Q1

lower=Q1-1.5*IQR
upper=Q3+1.5*IQR

outliers_iqr=df[(df['total_bill']<lower) | (df['total_bill']>upper)]

print("IQR Outliers:",len(outliers_iqr))
```

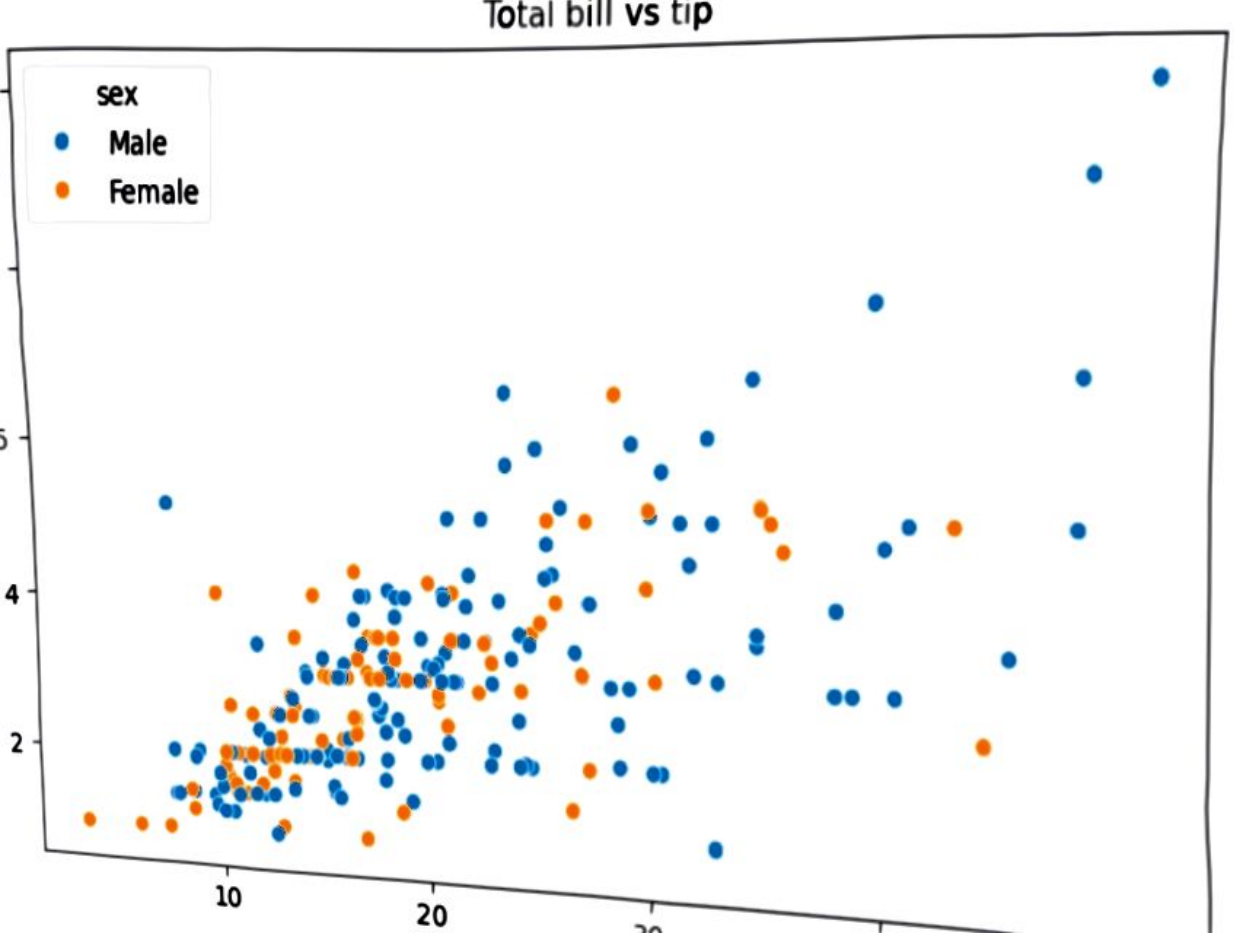
```
IQR Outliers: 9
```

Bivariate Analysis

- Bivariant Analysis means analyzing two variables together to understand the relationships between them.
- Bi = Two
- Variate = Variables

Examples

- Study hrs vs Exam Score
- Age vs Salary
- Total bill vs Fare

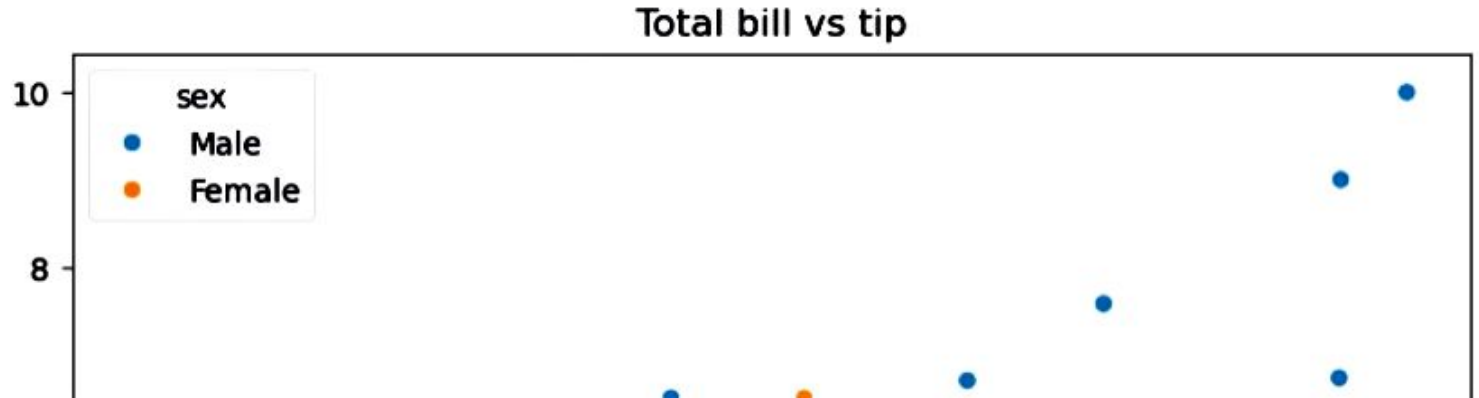

Total bill vs tip
sex
Male
Female
4
2
10
20

```
Types of Bivariate Analysis
  Variable1          variable2          visualization
  Numerical          Numerical            Scatter plot
  Categorical        Numerical            Bar chart
  Categorical        Categorical           Grouped Statistics/Crosstab
```

Numerical vs Numerical

- Used to determine whether one numerical variable affects another.
- Total bill vs tips

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='total_bill',
    y='tip',
    hue='sex',
    data=df
)
plt.title('Total bill vs tip')
plt.show()
```



Correlation

Pearson Correlation

[16]:
```
corr=df['total_bill'].corr(df['tip'])
print(corr)
```

```
0.6757341092113641
```

2.Categorical vs Numerical

Day vs total bill

- Compare average values across categories.

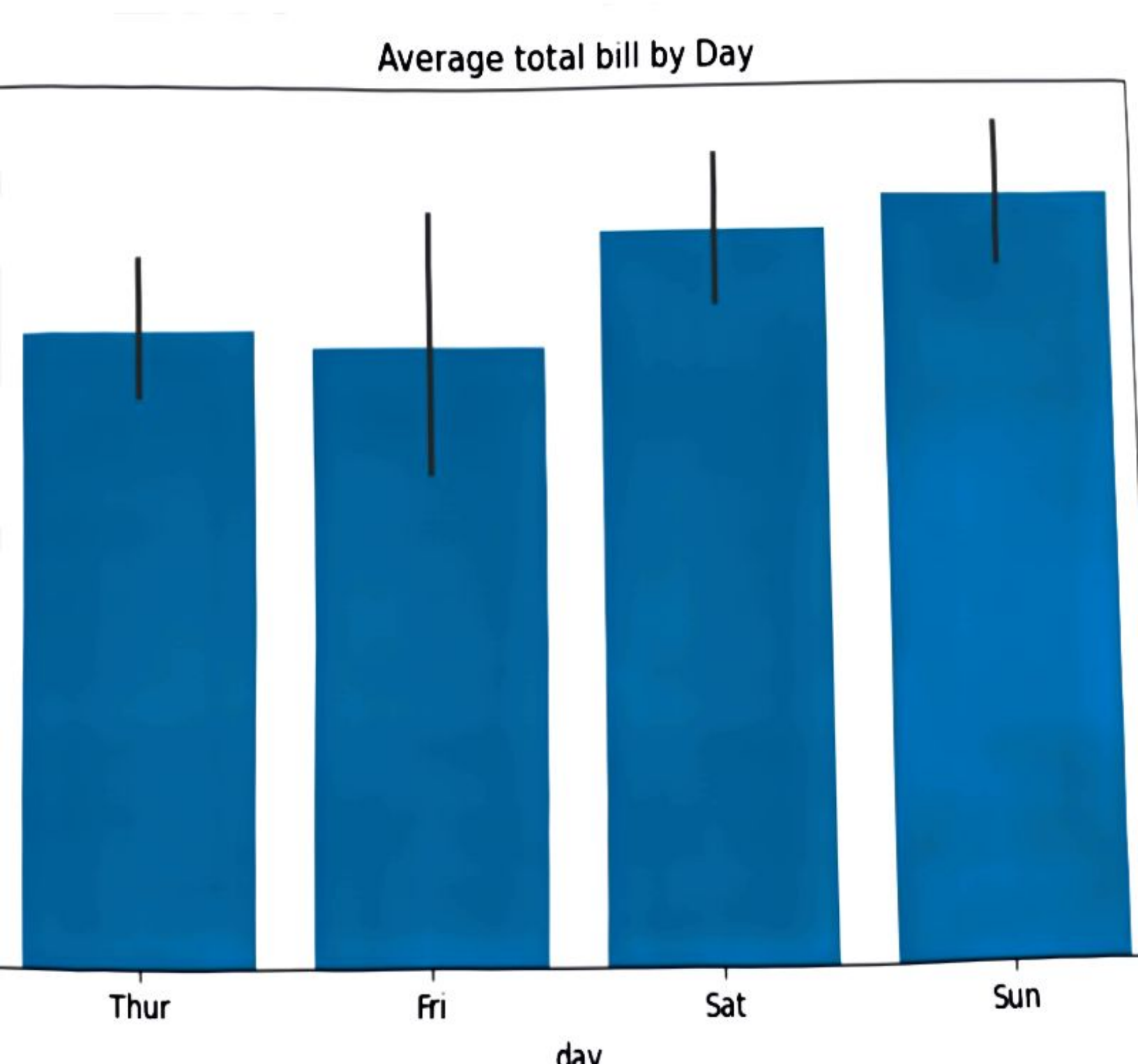
[18]:
```
#Bar chart
plt.figure(figsize=(8,5))

sns.barplot(
    x='day',
    y='total_bill',
    data=df
)
plt.title('Average total bill by Day')
plt.show()
```

```python
#Grouped Statistics
#Mean bill by day
print(
    df.groupby('day',observed=True)['total_bill'].mean()      #warning show so observed put bcz it combine both cate and num
)
#Multiple statistics
print(
    df.groupby('day',observed=False)['total_bill'].agg(['mean','median','min','max'])
)
```

```
day
Thur    17.682742
Fri     17.151579
Sat     20.441379
Sun     21.410000
Name: total_bill, dtype: float64
          mean  median   min    max
day
Thur  17.682742   16.20  7.51  43.11
Fri   17.151579   15.38  5.75  40.17
Sat   20.441379   18.24  3.07  50.81
Sun   21.410000   19.63  7.25  48.17
```

Sex vs Smoker

- Compare frequencies btw categories


Average total bill by Day
Thur
Fri
Sat
Sun
day

Multivariate analysis

- It means analyzing 3 or more variables simultaneously to understand complex relationships and patterns in data.

Example

Instead of studying:

- Height alone(Univariate)
- Height vs Weight(Bivariate)

We study:

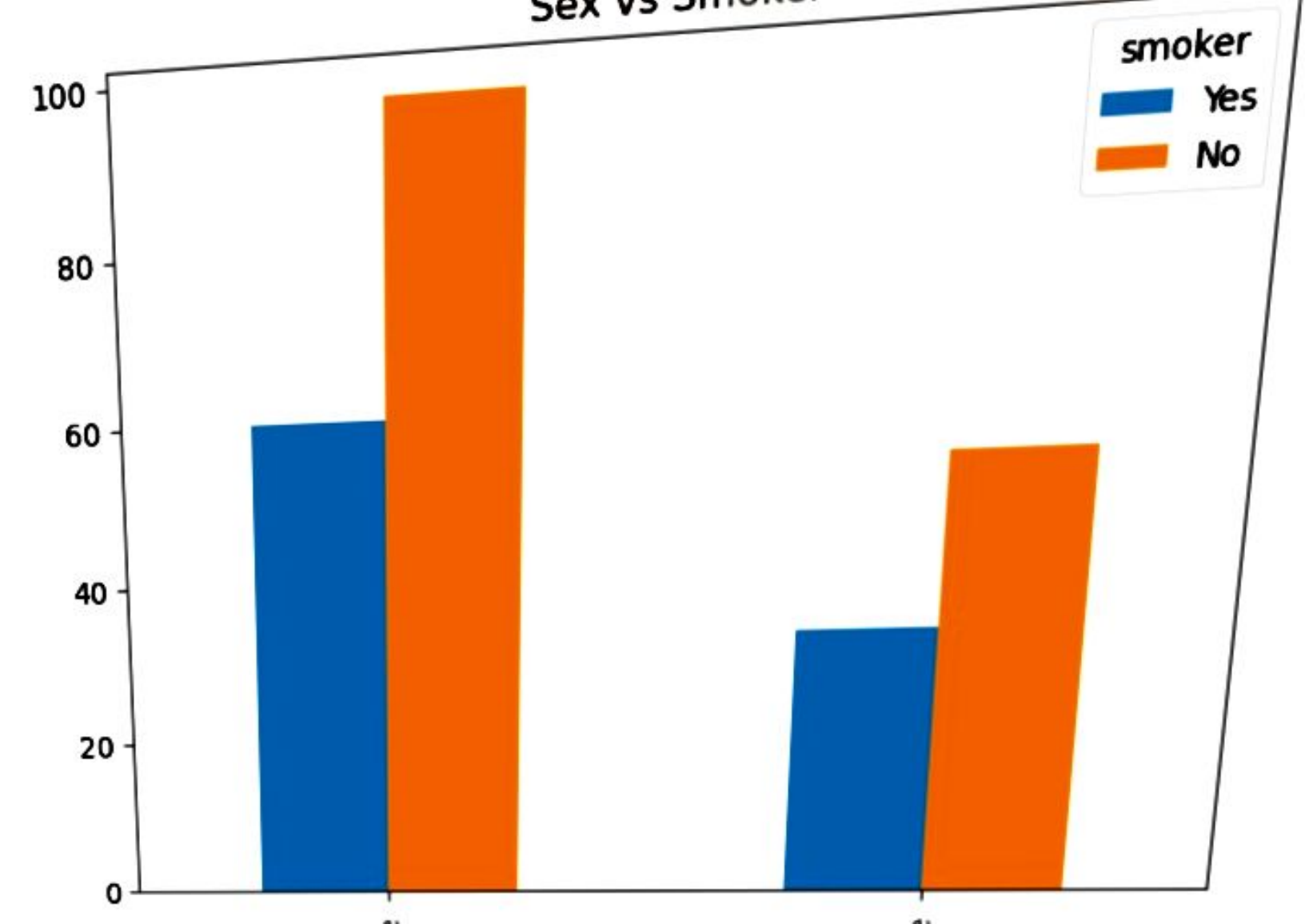
- Height vs Weight vs Age vs Gender.
- This is multivariate analysis.

Objectives of multivariate analysis

- Discover hidden relationships
- Understand interactions among multiple features
- Detect clusters and patterns
- Support feature selection
- Prepare data for ML

Pairplot

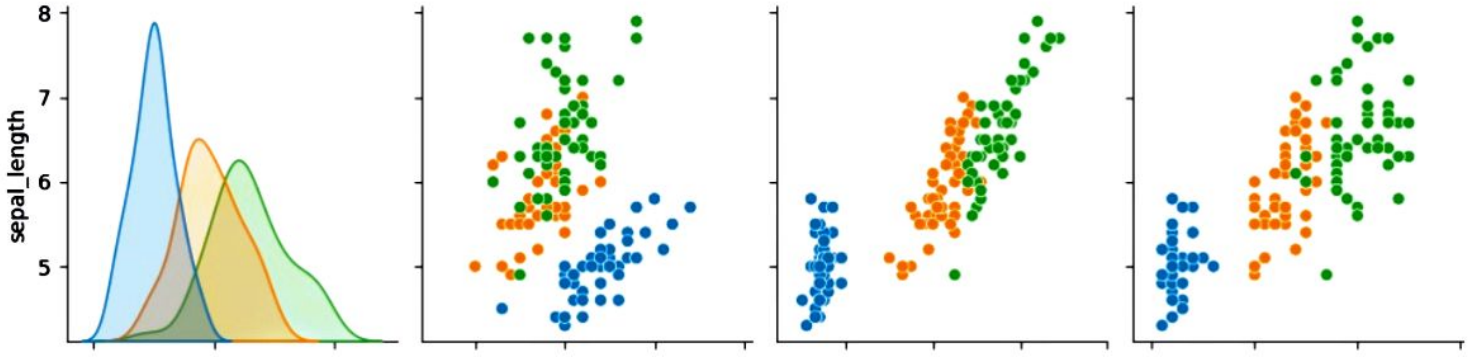
- It compares every numerical feature with every other numerical feature.
- It automatically creates:
- Scatter plots
- Histograms

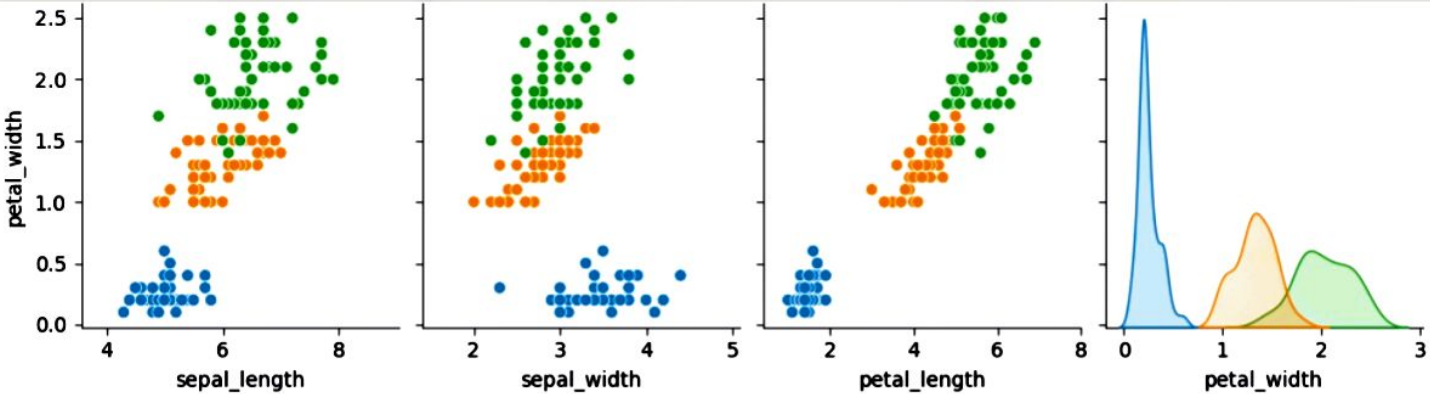

smoker
Yes
No
100
80
60
40
20
0

Why use pairplot?

- Detect correlation
- Find clusters

27]:
```
#pairplot
import seaborn as sns
import matplotlib.pyplot as plt

df=sns.load_dataset("iris")

sns.pairplot(
    df,
    hue='species'    #blue is separated but green and orange merge
)
plt.show()
```





2.Color-Coded Scatter Plot(3rd Dimension)

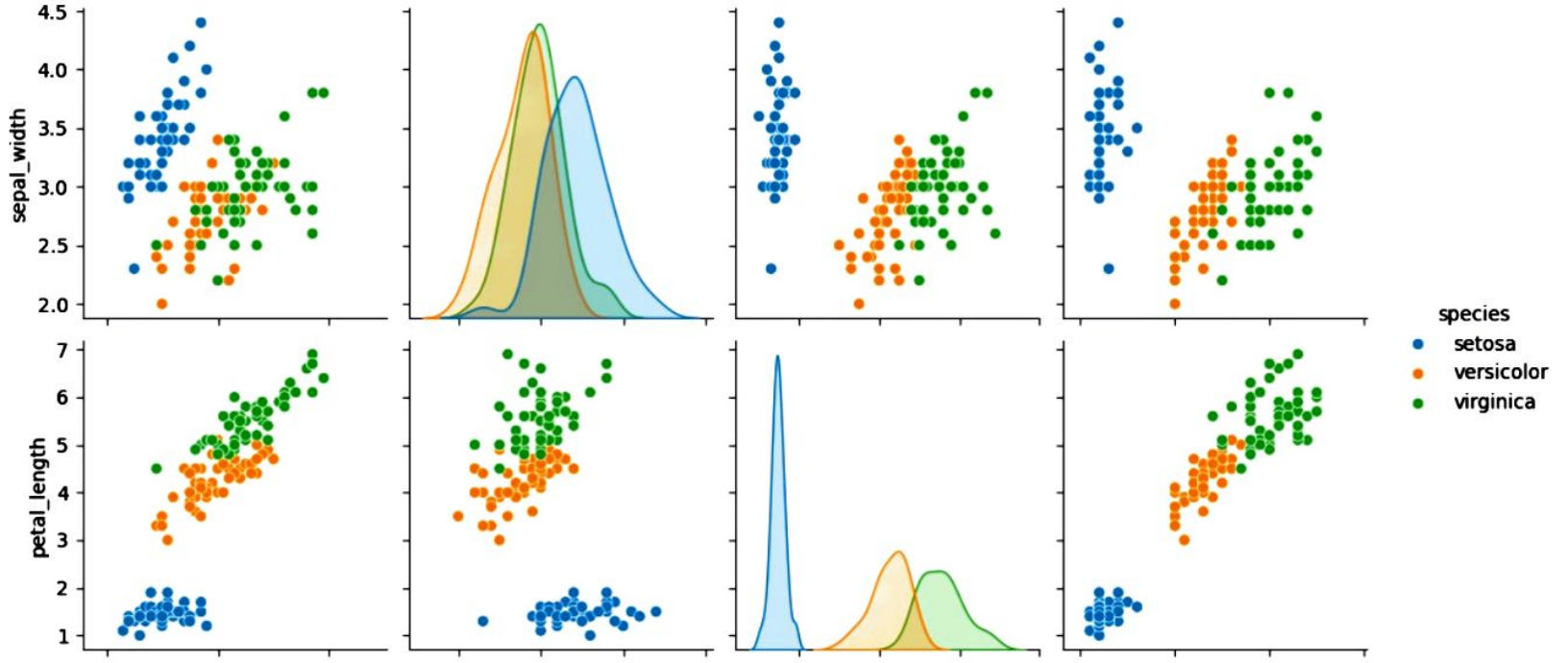
- A scatter plot normally shows:

1. X-axis -> Variable1
2. Y-axis ->Variable2

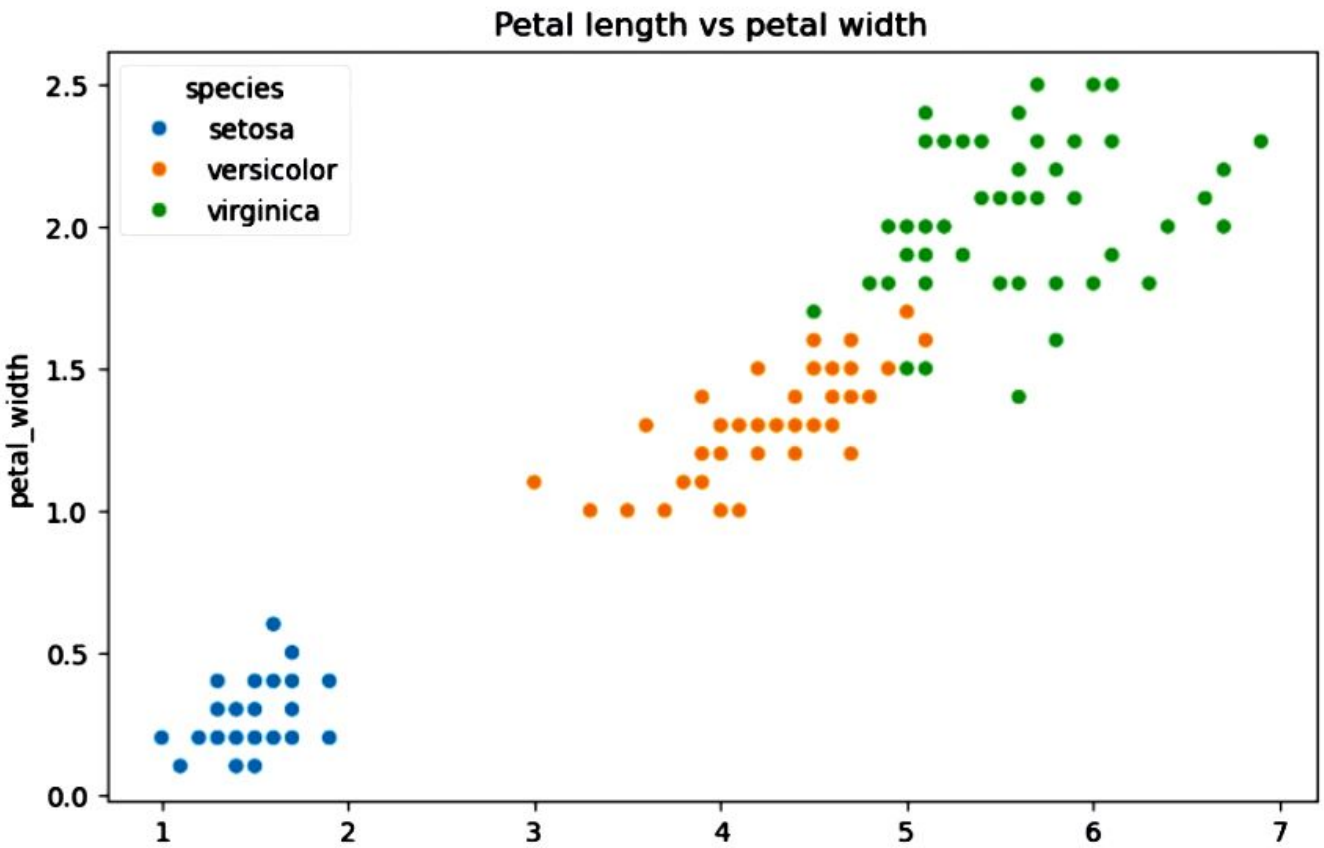
A third varuable can be represented using colors.

Example

1. X -> Petal Length
2. Y -> Petal Width
3. Color -> Species


sepal_width
petal_length
4.5
4.0
3.5
3.0
2.5
2.0
7
6
5
4
3
2
1
species
setosa
versicolor
virginica

```
plt.figure(figsize=(8,5))
sns.scatterplot(
    x='petal_length',
    y='petal_width',
    hue='species',
    data=df
)

plt.title("Petal length vs petal width")
plt.show()
```



3.Scatter plot

- A 3D scatter plot uses:

1. X-axis
2. Y-axis
3. 3D axis

```
#3D Scatter plot
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

fig=plt.figure(figsize=(8,6))

ax=fig.add_subplot(111,projection='3d')

ax.scatter(
    df['sepal_length'],
    df['sepal_width'],
    df['petal_length']
)

ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')
ax.set_zlabel('Petal length')
plt.show()
```

```python
#Example Dataset
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df=sns.load_dataset("iris")

print(df.head())
```

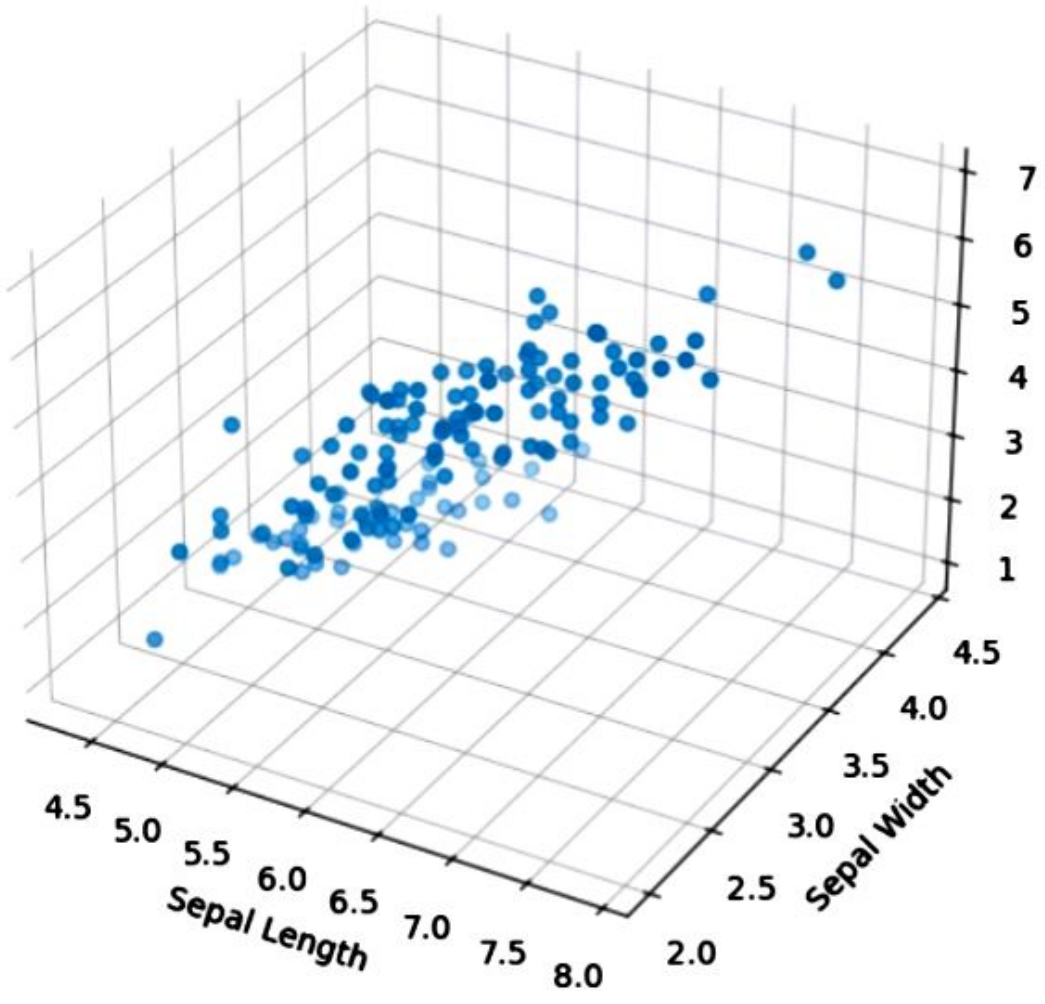
```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
```

Correlation matrix

- A correlation matrix shows the correlation btw all numerical columns.

```python
corr_matrix=df.corr(numeric_only=True)
print(corr_matrix)
```

```
              sepal_length  sepal_width  petal_length  petal_width
sepal_length      1.000000    -0.117570      0.871754     0.817941
sepal_width      -0.117570     1.000000     -0.428440    -0.366126
petal_length      0.871754    -0.428440      1.000000     0.962865
petal_width       0.817941    -0.366126      0.962865     1.000000
```



Correlation analysis

What is correlation?

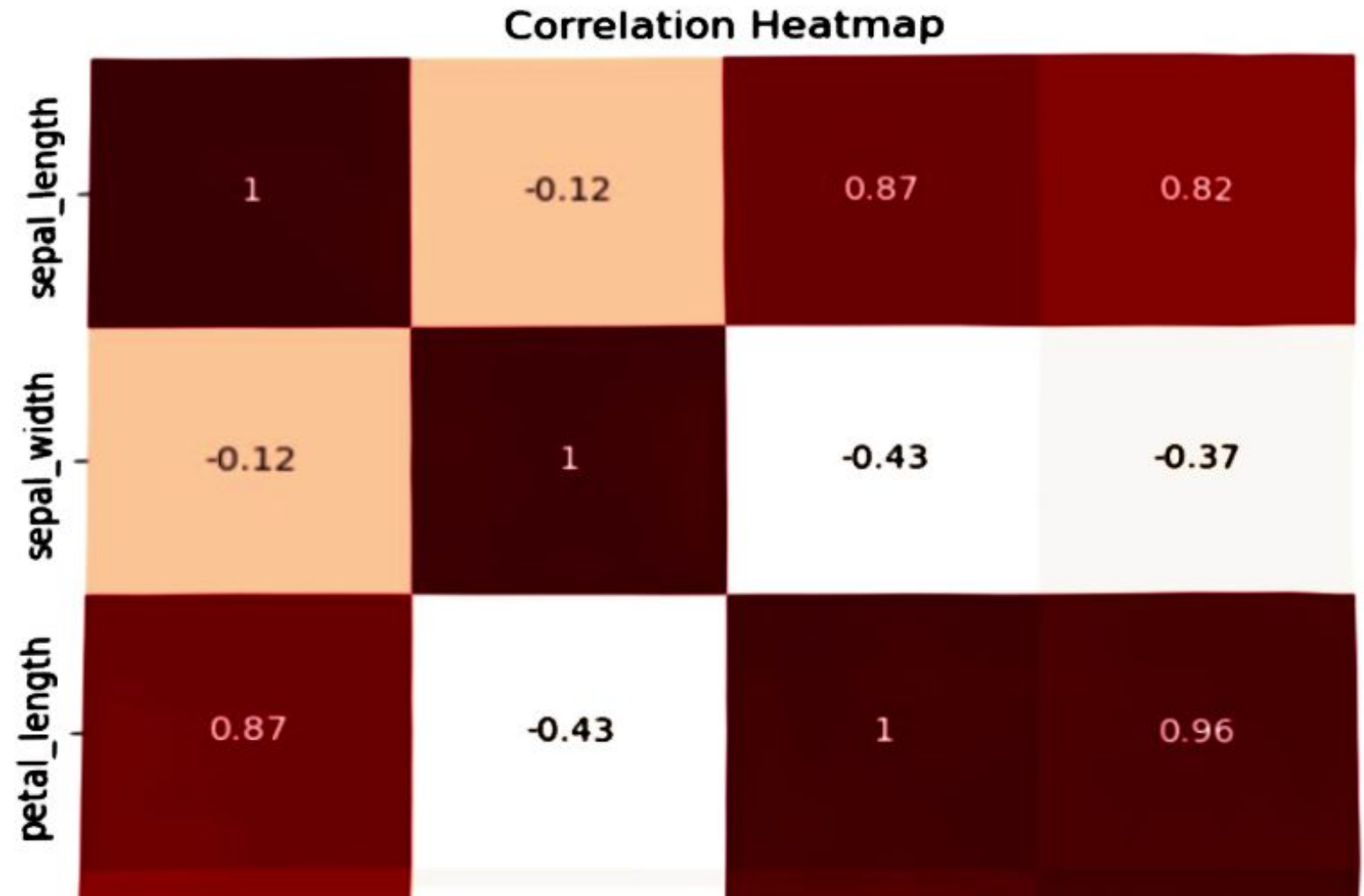
- Correlation measures the strength and direction of relationship btw 2 variables.

Correlation range

- -1<=r<=1

Heatmap

```
plt.figure(figsize=(8,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='Reds'
)

plt.title("Correlation Heatmap")
plt.show()
```



VIF(Variance Inflation Factor) for Multicollinearity Detection

What is multicollinearity?

- Multicollinearity occurs when independent variables(features) are highly correlated with each other.

Example

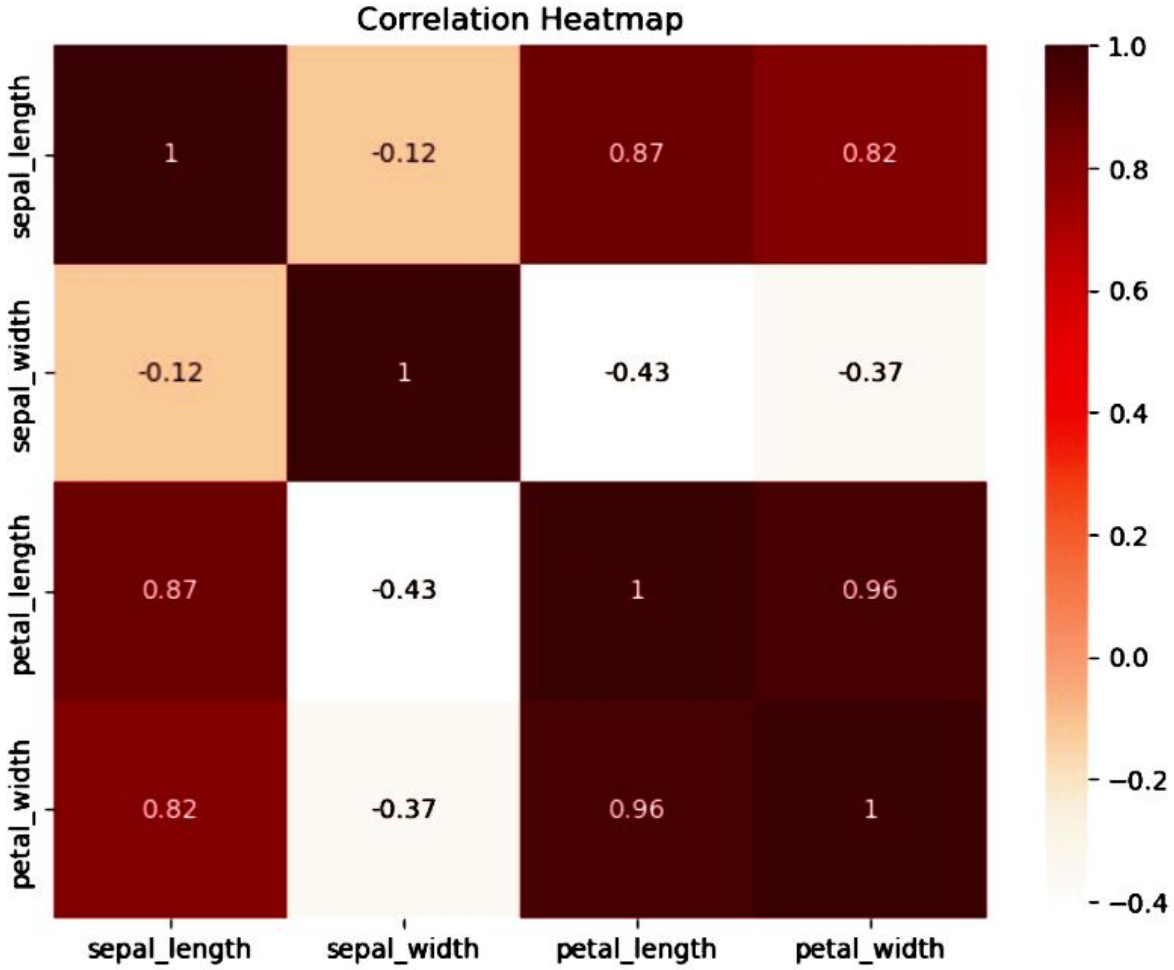
- Suppose a house price dataset contains.
    1. House area(sq.ft)
    2. No of rooms
    3. No of bedrooms
- These features may conatin similar information and be highly correlated.
- This creates multicollinearity.

What is VIF?

- Variance Inflation Factor(VIF) measures how much the variance of a feature is inflated bcz of its correlation with other features.

VIF Interpretation Table

VIF value	Interpretation
1	No correation
1-5	Low correlation
5-10	Moderate multicollinearity
>10	Severe multicollinearity



VIF(Variance Inflation Factor) for Multicollinearity Detection

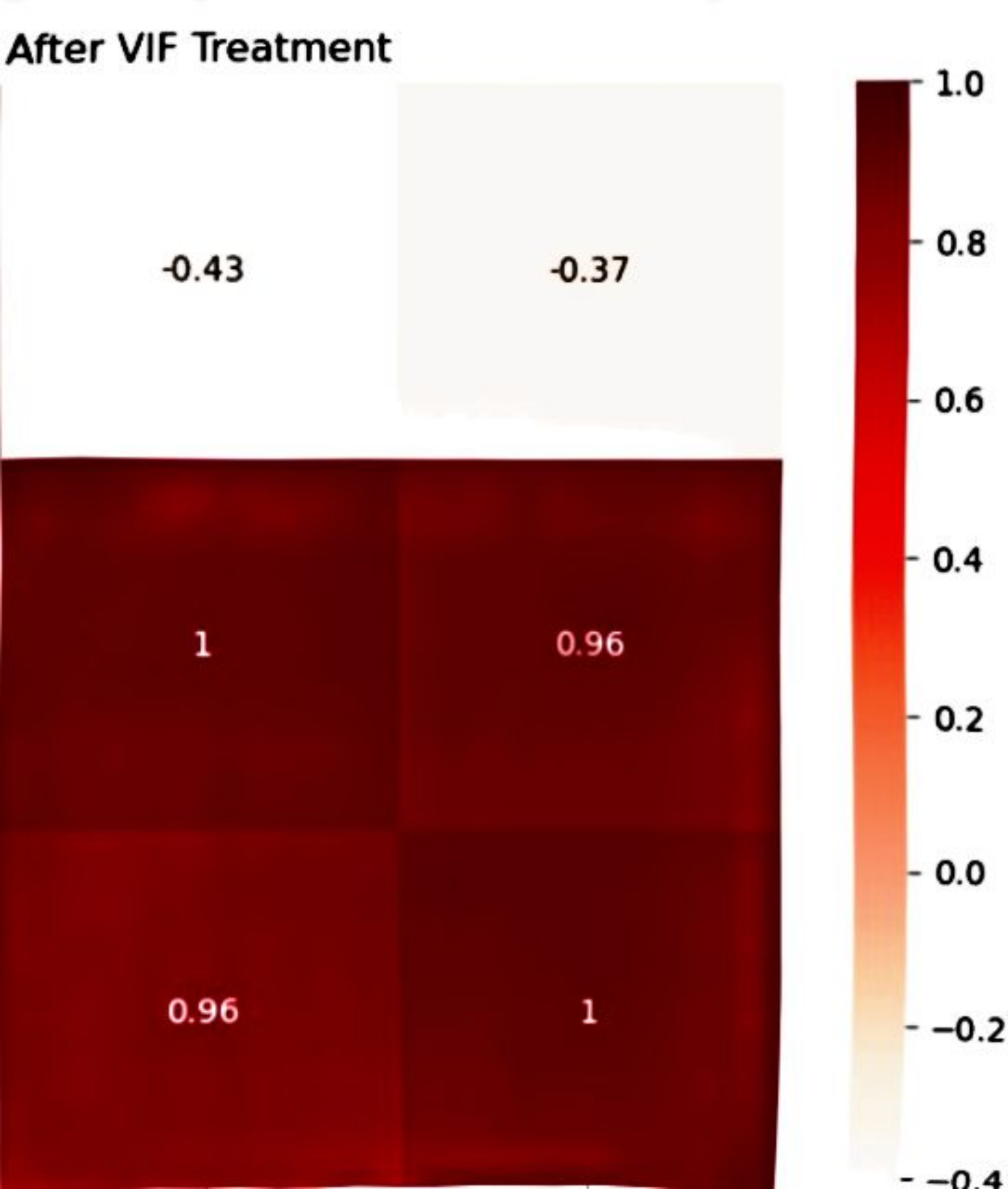
What is multicollinearity?

```
]: from statsmodels.stats.outliers_influence import variance_inflation_factor
X= df.drop("species",axis=1)
vif = pd.DataFrame()

vif["Feature"] = X.columns

vif["VIF"] = [
    variance_inflation_factor(X.values,i)
    for i in range(X.shape[1])
]
print(vif)
```

```
        Feature         VIF
0  sepal_length  262.969348
1   sepal_width   96.353292
2  petal_length  172.960962
3   petal_width   55.502060
```

```
]: #Remove a High-VIF Feature
#Suppose we remove petal_length.

X_new=X.drop("sepal_length",axis=1)

vif_after=pd.DataFrame()

vif_after["Feature"]=X_new.columns

vif_after["VIF"]=[
    variance_inflation_factor(X_new.values,i)
    for i in range(X_new.shape[1])]
print(vif_after)
```

After VIF Treatment



	Feature	VIF
0	sepal_width	5.856965
1	petal_length	62.071308
2	petal_width	43.292574

```
#Heatmap after removing feature
plt.figure(figsize=(8,6))

sns.heatmap(
    X_new.corr(),
    annot=True,
    cmap='Reds'
)

plt.title("After VIF Treatment")
plt.show()
```